TRƯỜNG ĐẠI HỌC ĐỒNG THÁP
Khoa Công nghệ và Kỹ thuật

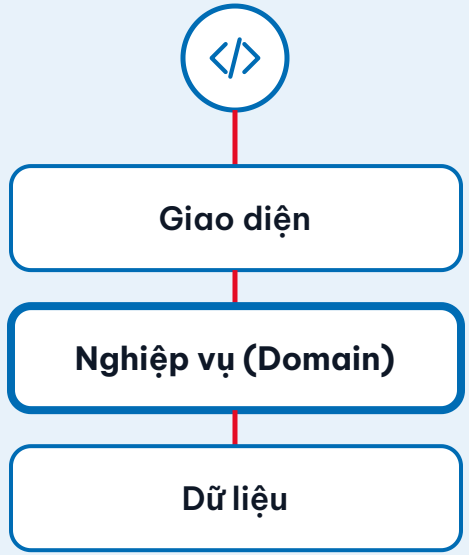
IN4211 · CHƯƠNG 1

Giới thiệu phần mềm & môi trường

Phát triển phần mềm hướng đối tượng

Giảng viên **ThS. Nguyễn Tuấn Kiệt**
Năm học 2026–2027

C# .NET OOP Git




```
class Club {
string Name;
void Join(Student s);
}
```

0836 980 284 nguyentuankiet@dthu.edu.vn

Nội dung chương 1

🧰 **1.1** — .NET 10 ecosystem, công cụ phát triển, cài môi trường

🏛️ **1.2** — Ôn tập OOP cơ bản: class, inheritance, polymorphism, interface

🎓 **1.3** — Giới thiệu domain bài toán EduHub

🗂️ **1.4** — Tư duy mô hình hoá lớp (class diagram sơ bộ)

🌿 **1.5** — Git và GitHub cơ bản · quy ước commit message

📝 **Bài tập Ch01** — Phân tích yêu cầu và Domain Modeling EduHub

Học xong môn này bạn làm được gì? (1/2)

CLO = chuẩn đầu ra: việc bạn làm được khi học xong. Môn có 7 CLO:

- **CLO1** — Thiết kế và xây phần mềm theo Clean Architecture (4 lớp) trên .NET 10; áp dụng SOLID và chọn đúng Design Pattern
- **CLO2** — Xây REST API an toàn (JWT, Argon2id), dùng ORM (EF Core, Dapper) với PostgreSQL; quản lý code bằng Git/GitHub theo quy trình nhóm
- **CLO3** — Viết unit test (xUnit, Moq, coverage ≥ 50%), đóng gói bằng Docker, thiết lập CI/CD cơ bản với GitHub Actions
- **CLO4** — Vận dụng kiến thức để xây một ứng dụng web quản lý hoàn chỉnh (EduHub): từ phân tích yêu cầu, thiết kế đến triển khai

Học xong môn này bạn làm được gì? (2/2)

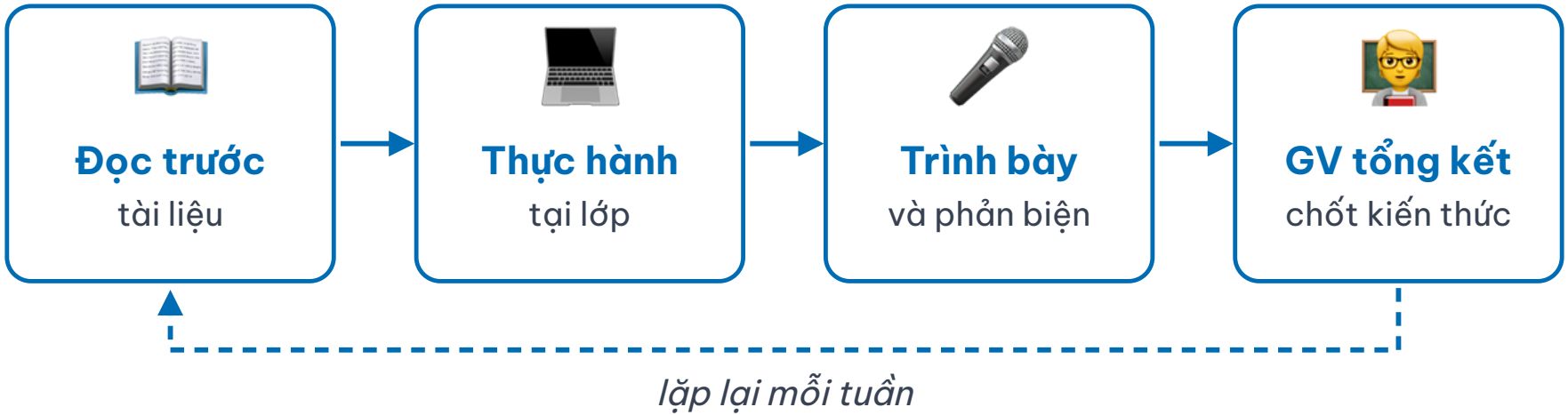
- **CLO5** — Làm việc nhóm theo quy trình Agile; trình bày kết quả kỹ thuật rõ ràng và nhận phản hồi mang tính xây dựng
- **CLO6** — Thực hiện nghiêm nội quy lớp học và quy định của học phần; tham dự đủ buổi (vắng không quá 20% số tiết)
- **CLO7** — Tự tìm hiểu, tự nghiên cứu, tiếp cận công nghệ mới từ nhiều nguồn để tạo ra sản phẩm theo yêu cầu

Chương 1 góp phần vào CLO1, CLO4 và CLO5: mô hình hoá EduHub bằng class, làm quen Git/GitHub và nộp bài bằng Pull Request.

Lộ trình 15 tuần

TUẦN	PHẦN	NỘI DUNG
1–2	**Chương 1**	Giới thiệu, OOP, .NET 10, Git, Domain Modeling
3–4	**Chương 2**	SOLID, Design Patterns, unit test, Git nâng cao
5–6	**Chương 3**	Clean Architecture, DI, REST API, Swagger
6–11	**Chương 4**	EF Core, PostgreSQL, Dapper, CRUD đầy đủ
11–13	**Chương 5**	JWT, Argon2id, phân quyền, OWASP, QR
13–15	**Chương 6**	Kiểm thử, Docker, GitHub Actions, demo EduHub
4, 9, 12, 14	**SV thuyết trình**	DDD · Microservices · OAuth2/OIDC · DevOps

Cách lớp học vận hành



- **Chương 1–2:** giảng viên giới thiệu tổng quan và khởi động dự án EduHub
- **Từ Chương 3:** sinh viên tự nghiên cứu, **nhóm 2–3 SV** thuyết trình; giảng viên phản biện và chuẩn hoá kiến thức

Quy trình nộp slide thuyết trình

1. **Nộp slide** trước 23:59 của ngày cách buổi thuyết trình **4 ngày**: push lên nhánh của nhóm, tạo Pull Request về `main`, tag giảng viên
2. **Giảng viên phản hồi** trên Pull Request trong vòng **24 giờ**
3. **Nhóm chỉnh sửa**, push commit mới trước buổi học tối thiểu **2 giờ**
4. **Thuyết trình và phản biện** tại lớp; slide cuối được link vào issue tương ứng

> Nộp trễ: không có phản hồi trước và bị trừ 30% điểm thuyết trình. Không có commit chỉnh sửa: mất 10% điểm phần phản hồi.

Cách chấm điểm

MÃ	THÀNH PHẦN	TỶ TRỌNG
HĐ1.1	Chuyên cần (điểm danh)	20% của HĐ1
HĐ1.2	Thuyết trình nhóm	40% của HĐ1
HĐ1.3	Bài tập chương (nộp Pull Request)	40% của HĐ1
HĐ2.1	Bài tập lớn: demo, báo cáo, commit history	50% tổng điểm

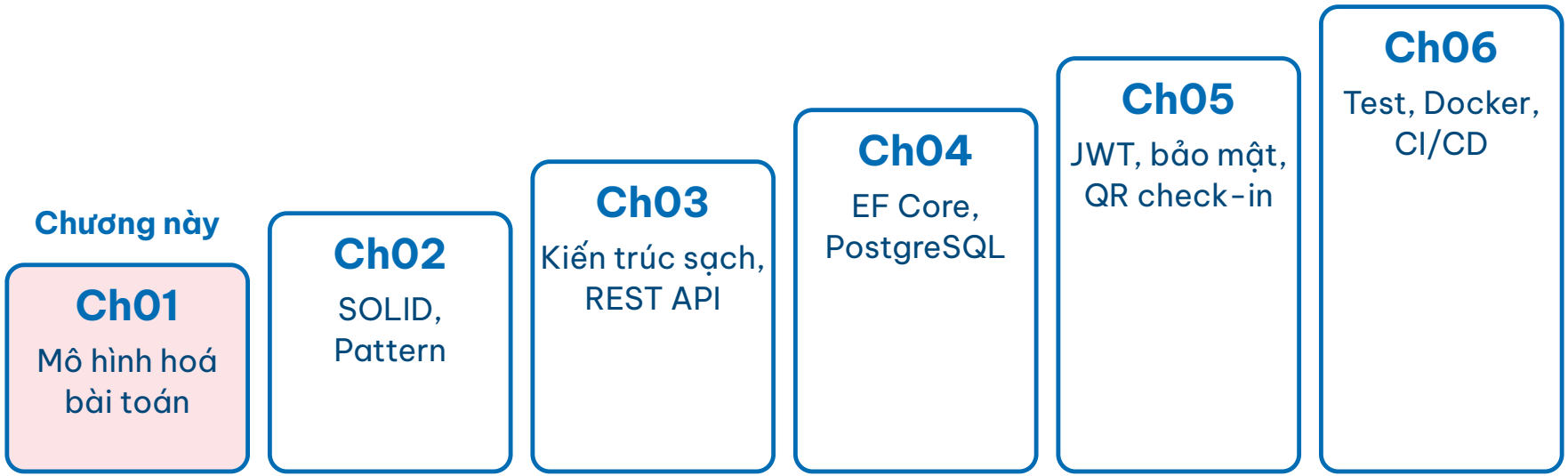
HĐ1 (đánh giá thường xuyên) = 50% tổng điểm · **HĐ2.1 = 50%**. Chương 1 có bài tập chấm theo HĐ1.3.

Quy định đối với sinh viên

- Tham dự đủ từ **80%** số tiết; vào lớp trễ 15 phút tính là vắng buổi đó
- Không làm việc riêng, không dùng điện thoại trong giờ học
- Tài khoản trực tuyến định danh bằng **MSSV**, họ tên thật, avatar thật
- Thuyết trình theo nhóm 2–3 SV; chủ đề và thứ tự **bốc thăm** ở buổi đầu, ghi vào Google Sheets và commit vào repo của lớp trong 24 giờ
- Bài tập lớn: mỗi tính năng một commit, message theo **Conventional Commits**; Pull Request phải có mô tả (what, why, how to test)

EduHub — dự án xuyên suốt học kỳ

EduHub = hệ thống quản lý câu lạc bộ (CLB) và sự kiện sinh viên. Mỗi chương thêm một phần mới vào cùng dự án.



1

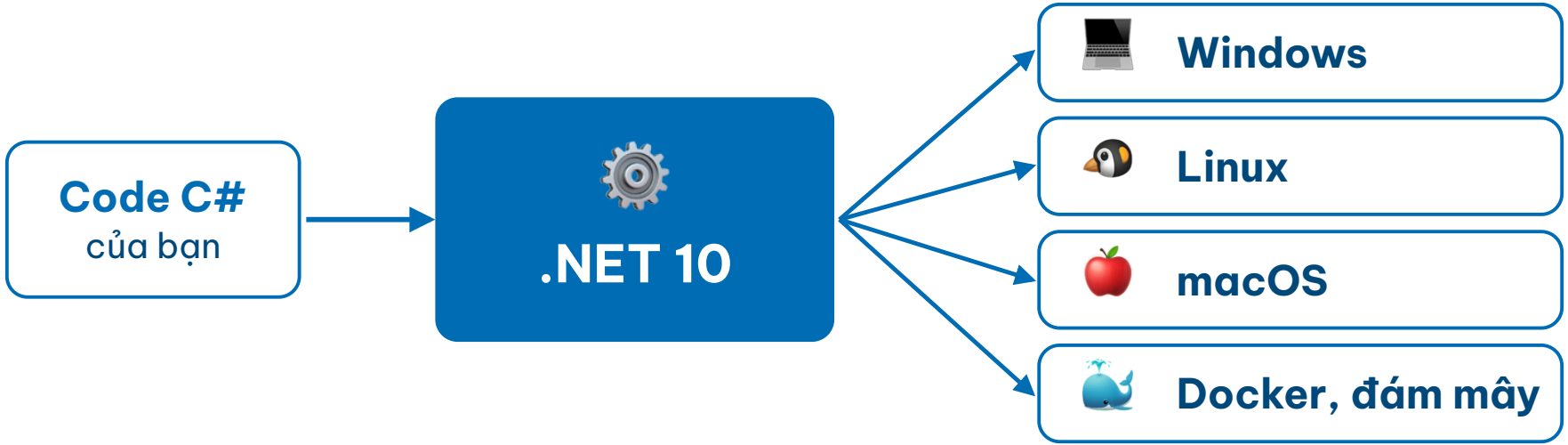
PHẦN 1 / 6

PHẦN 1

1.1 · .NET 10 ecosystem, công cụ và cài môi trường

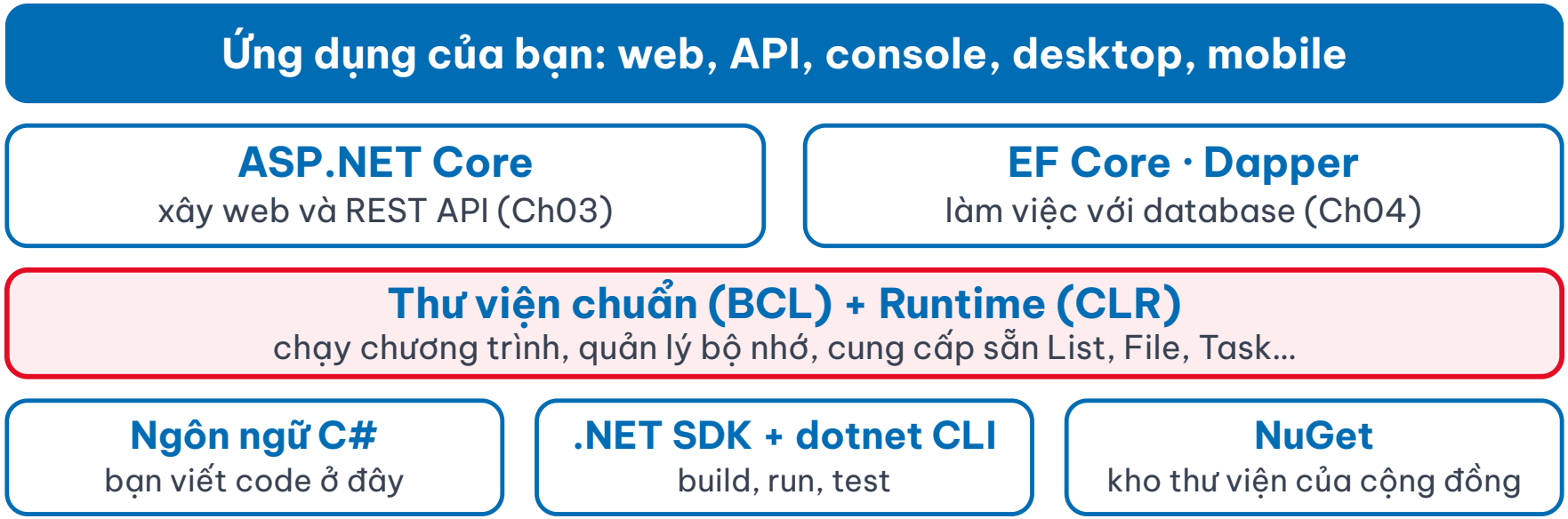
- .NET 10 là gì, vì sao chọn
- Hệ sinh thái .NET 10
- VS Code, JetBrains Rider và công cụ cần cài
- Cài, kiểm tra và chạy chương trình đầu tiên

Tại sao chọn .NET 10?



🌍 Viết một lần, chạy trên mọi hệ điều hành

🧰 Làm được web, API, mobile, desktop



Hình 1.1. Các thành phần chính của hệ sinh thái .NET 10 (tự vẽ).

Tại sao chọn .NET 10? (tiếp)

🚀 **Chạy nhanh** — luôn nằm nhóm đầu các bảng so tốc độ web

🏢 **Nhiều nơi dùng** — Microsoft, FPT Software, VPBank Tech

🎓 **LTS** = bản được Microsoft hỗ trợ lâu dài. .NET 10 được cập nhật đến 11/2028

📅 Học một lần, dùng đến khi tốt nghiệp

Công cụ cần cài (1/2) — Viết code

.NET 10 SDK

Bộ công cụ viết & chạy C# (bắt buộc)

dotnet.microsoft.com/download

Visual Studio Code

Trình soạn code nhẹ, miễn phí. Cài thêm tiện ích "C# Dev Kit"

code.visualstudio.com/download

JetBrains Rider

IDE cho .NET, chạy cả Windows/macOS/Linux. Sinh viên dùng miễn phí

jetbrains.com/rider/download

Git

Lưu lịch sử thay đổi của code (học ở mục 1.5)

git-scm.com/downloads

Chỉ cần chọn **một** trong ba: VS Code, Visual Studio hoặc Rider.

1.5 Công cụ cần cài (2/2) — Dùng ở các chương sau





PostgreSQL

Cơ sở dữ liệu dùng từ Chương 4. Bộ cài kèm pgAdmin

postgresql.org/download



Postman

Gọi thử API: gửi yêu cầu (request), xem kết quả trả về (response)

postman.com/downloads



Docker Desktop

Chạy PostgreSQL & app trong "container" — hộp đóng gói sẵn (Chương 6)

docker.com/get-started



Tài khoản GitHub

Lưu code trên mạng, làm việc nhóm, nộp bài bằng Pull Request

github.com/signup

Chương 1–2 chưa cần các công cụ này — cài trước Chương 3 (Postman) và Chương 4 (PostgreSQL).

Cài và kiểm tra

```
winget install Microsoft.DotNet.SDK.10   # cài .NET (Windows)
dotnet --version                         # kiểm tra .NET
git --version                            # kiểm tra Git
git config --global user.name "Tên bạn"  # tên hiện trong lịch sử Git
git config --global user.email "email@truong.edu.vn"  # email của bạn
```

- Tài khoản GitHub: dùng **MSSV**, họ tên thật và ảnh thật (quy định của lớp)
- Tạo repo (kho chứa code) `eduhub` để làm bài tập chương

1.7 Hello World — chạy chương trình đầu tiên

1 Tạo thư mục

```
mkdir EduHub
cd EduHub
```

2 Tạo project

```
dotnet new console
-n EduHub.Console
```

3 Chạy thử

```
cd EduHub.Console
dotnet run
```

Console app = chương trình chạy trong cửa sổ dòng lệnh. `mkdir` = tạo thư mục, `cd` = đi vào thư mục.

Nội dung Program.cs

```
// EduHub.Console/Program.cs
Console.WriteLine("Chào mừng đến với EduHub!");   // in ra màn hình
Console.WriteLine($"Hôm nay: {DateTime.Now:dd/MM/yyyy}");
```

```
$ dotnet run
Chào mừng đến với EduHub!
Hôm nay: 05/10/2026
```

ACTIVE RECALL Lệnh `dotnet new console` tạo ra những file nào?

Cấu trúc project .NET

```
EduHub/
├── EduHub.sln                  ← gom nhiều project
├── EduHub.Console/
│   ├── EduHub.Console.csproj   ← mô tả project
│   ├── Program.cs              ← điểm bắt đầu chạy
│   └── obj/                    ← file tạm (đừng commit)
└── EduHub.Domain/
    ├── EduHub.Domain.csproj    ← thư viện chứa các class của bài toán
    └── Entities/Member.cs
```

Ba file chính: **.sln** (gom nhiều project), **.csproj** (khai báo .NET và thư viện), **Program.cs** (điểm bắt đầu chạy).

2

PHẦN 2 / 6

PHẦN 2

1.2 · Ôn tập OOP cơ bản

- 4 trụ cột của OOP
- Class, kế thừa (inheritance) và đa hình (polymorphism)
- Encapsulation — cách sai và cách đúng
- Interface hay Abstract class

4 trụ cột của OOP



Encapsulation (Đóng gói)

Giấu dữ liệu trong class,
chỉ mở cửa qua method.

Như máy ATM: chỉ bấm nút



Abstraction (Trừu tượng)

Chỉ nói "làm gì", giấu
"làm như thế nào".

Như lái xe: không cần hiểu động cơ



Inheritance (Kế thừa)

Class con dùng lại của cha,
rồi thêm cái mới.

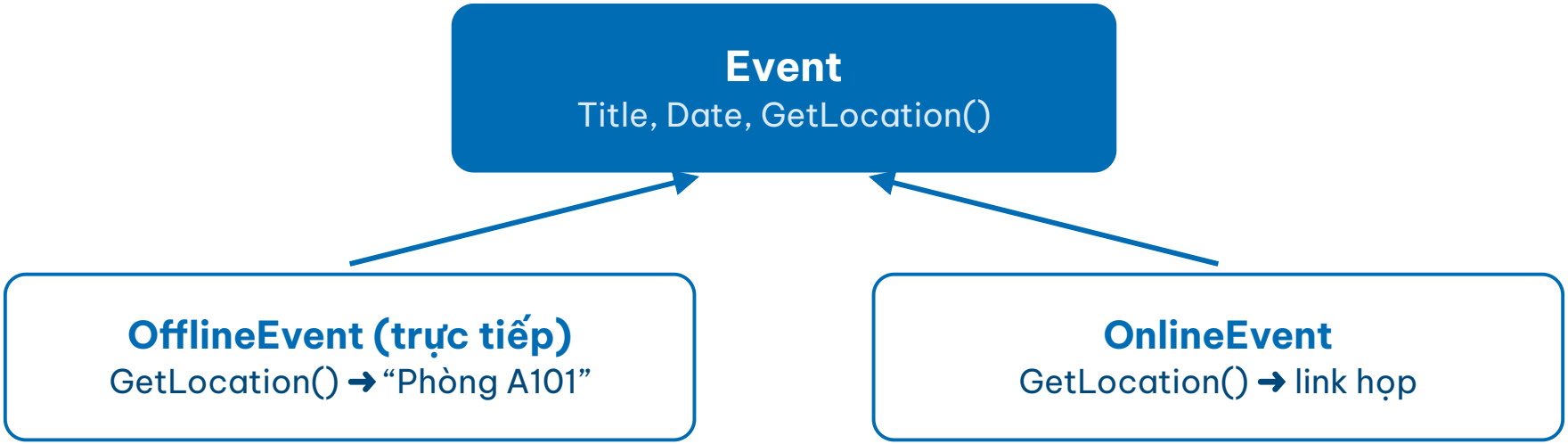
Như con thừa hưởng cha mẹ



Polymorphism (Đa hình)

Cùng một lệnh, mỗi loại
làm theo cách riêng.

Như "kêu": chó sủa, mèo meo



Cùng gọi `GetLocation()`, mỗi loại event trả kết quả riêng.

Data bag — cách làm sai

```
public class Member
{
    public string Email { get; set; }      // ai cũng sửa được
}

member.Email = "abc";           // email sai mà vẫn nhận
club.Members.Add(member);       // thêm thẳng, bỏ qua luật chống trùng
member.Status = "???";          // gõ bậy gì cũng nhận
```

Mọi luật nghiệp vụ đều bị **qua mặt**.

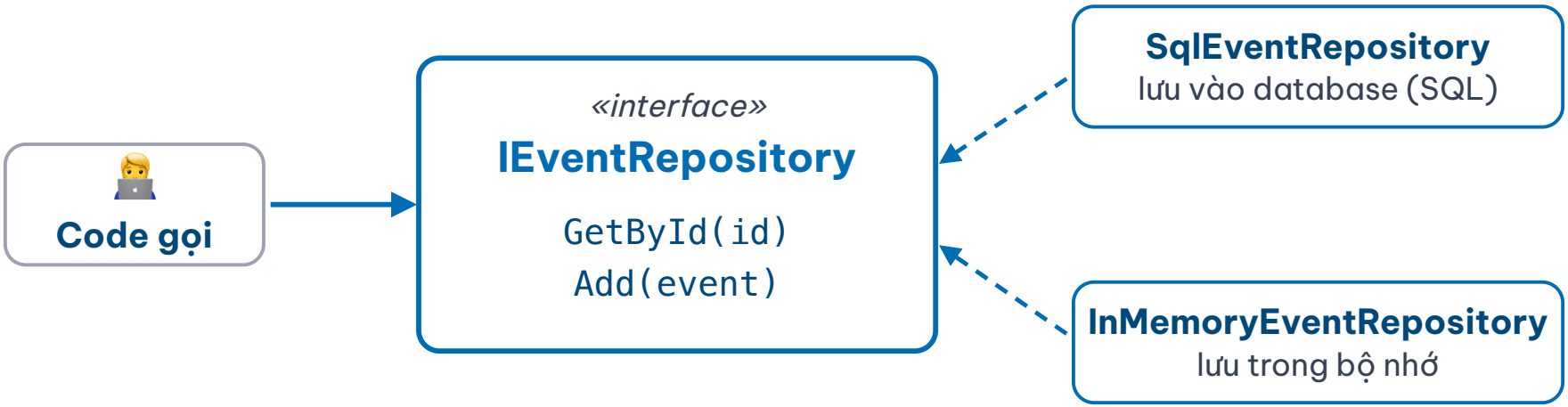
Encapsulation — cách làm đúng

```
public string Email { get; private set; }

public static Member Create(/* ... */)
{
    if (!email.Contains('@'))
        throw new ArgumentException("Email sai");
    // ...
}

public IReadOnlyList<Member> Members => _members;   // chỉ xem
public void AddMember(Member m) { … }               // chặn trùng
```

2.5 Interface = “hợp đồng”



Chỉ nói **làm được gì**. Như ổ cắm: thiết bị nào đúng chuẩn cũng cắm được. Mũi tên đứt = “làm theo hợp đồng”.

Interface hay Abstract class?

🔌 Interface

- Hợp đồng: **làm được gì**
- Không có code làm sẵn
- Một class làm theo được **nhiều** interface
- Dễ thay bằng bản giả (mock) để chạy thử

🏗️ Abstract class

- Chia sẻ **code dùng chung**
- Có method viết sẵn và method để trống cho class con tự viết
- Mỗi class chỉ kế thừa **một** class cha
- Quan hệ “là một loại” (is-a)

3

PHẦN 3 / 6

PHẦN 3

1.3 · Giới thiệu domain bài toán EduHub

- Bài toán và bức tranh tổng thể
- User Story và tiêu chí nghiệm thu
- Ba vai trò (actor) của EduHub

Bài toán EduHub

EduHub giúp sinh viên tham gia CLB và dự sự kiện. Mỗi đối tượng chính của bài toán gọi là **entity**:

- Sinh viên (**Member**) xem và tham gia **Club**
- Mỗi Club tổ chức nhiều **Event**: workshop, seminar, hackathon
- Đăng ký dự sự kiện → tạo **Registration** (phiếu đăng ký)
- Ngày diễn ra: ban tổ chức ghi **CheckIn** (điểm danh)

Bức tranh tổng thể



Chương 1: chỉ viết class OOP, chưa có database, chưa có API.

Mỗi hình vuông sẽ thành **một class C#**. Cả bộ class này gọi là **domain model** (mô hình của bài toán).

User Story là gì?

User Story (câu chuyện người dùng) = một yêu cầu viết bằng câu đơn giản
"Là [vai trò], tôi muốn [việc cần làm], để [lợi ích]."

Tiêu chí nghiệm thu (acceptance criteria) = điều kiện để biết đã làm xong

Given (cho trước)	→	When (khi)	→	Then (thì)
tình huống bắt đầu		người dùng làm gì		kết quả mong đợi

User Story — Sinh viên

US-01 · Sinh viên: tham gia một CLB

① Chưa là thành viên ➜ tham gia ➜ có tên trong `club.Members`
② Đã là thành viên ➜ tham gia lại ➜ bị từ chối (không trùng)

US-02 · Sinh viên: đăng ký một sự kiện

① Còn chỗ ➜ đăng ký ➜ tạo `Registration` (Pending = chờ xác nhận)
② Hết chỗ (đủ `Capacity` = sức chứa) ➜ đăng ký ➜ báo hết chỗ

US-03 · Sinh viên: check-in (điểm danh) tại sự kiện

① Phiếu Confirmed ➜ check-in ➜ tạo `CheckIn`
② Phiếu Pending / Cancelled ➜ check-in ➜ bị từ chối

User Story — Ban CLB & Admin

US-04 · Ban CLB: tạo sự kiện

① `Capacity > 0`, có tiêu đề ➜ tạo ➜ gắn đúng `ClubId`
② Tiêu đề trống hoặc capacity ≤ 0 ➜ tạo ➜ báo lỗi

US-05 · Ban CLB: xem báo cáo tham dự

① Thấy số người đăng ký / sức chứa và số người đã check-in
② Liệt kê từng người: đã / chưa check-in

US-06 · Admin: ngừng một thành viên

① Thành viên đang hoạt động (Active) ➜ ngừng ➜ `IsActive()` = false
② Lịch sử đăng ký của người đó vẫn được giữ

3.6 3 vai trò (actor) của EduHub



Sinh viên

US-01 → US-03

tham gia CLB, đăng ký, check-in



Ban CLB

US-04 → US-05

tạo sự kiện, xem báo cáo



Admin

US-06

quản lý thành viên

Bài tập: tự viết ít nhất 5 story, đủ 3 vai trò. Đừng chép story mẫu!

4

PHẦN 4 / 6

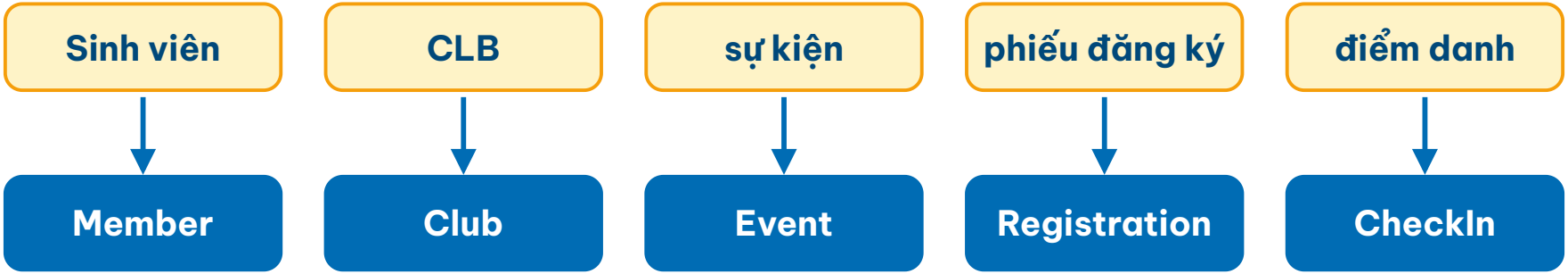
PHẦN 4

1.4 · Tư duy mô hình hoá lớp

- Từ User Story tìm ra class
- Sơ đồ lớp (class diagram) sơ bộ
- Quan hệ và khoá ngoại
- Từ sơ đồ đến code C#

Từ User Story tìm ra Entity

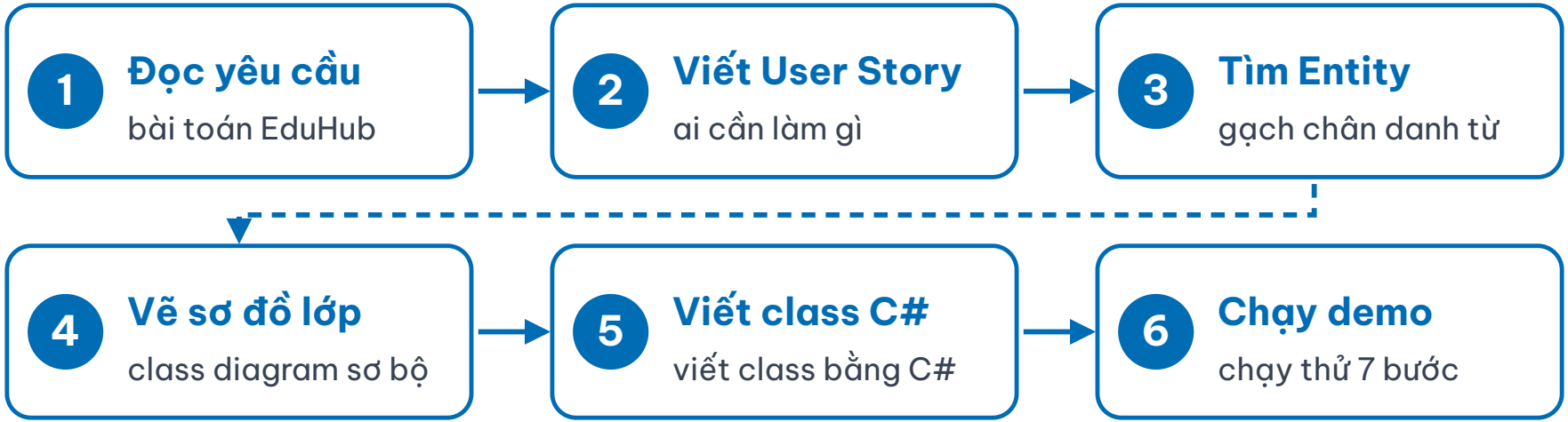
Danh từ trong các story:



Mỗi danh từ chính thành một entity (class).

ACTIVE RECALL *Tiêu đề* và *sức chứa* cũng là danh từ trong story. Vì sao chúng **không** thành entity riêng?

4.2 Từ yêu cầu đến code — 6 bước



5 Entity – thuộc tính & hành vi (1)

👤 Member – thành viên

Thuộc tính: `Id, Name, Email, StudentCode, JoinedAt, Status`
Hành vi (method): `Create()`, `IsActive()`, `Deactivate()`

🏛️ Club – câu lạc bộ

Thuộc tính: `Id, Name, Description, CreatedAt, Members`
Hành vi (method): `AddMember()` (chặn trùng), `MemberCount()`

📅 Event – sự kiện

Thuộc tính: `Id, ClubId (khoá ngoại), Title, Date, Capacity, IsOnline`
Hành vi (method): `HasCapacity()` – còn chỗ không?

5 Entity — thuộc tính & hành vi (2)

📋 Registration — phiếu đăng ký

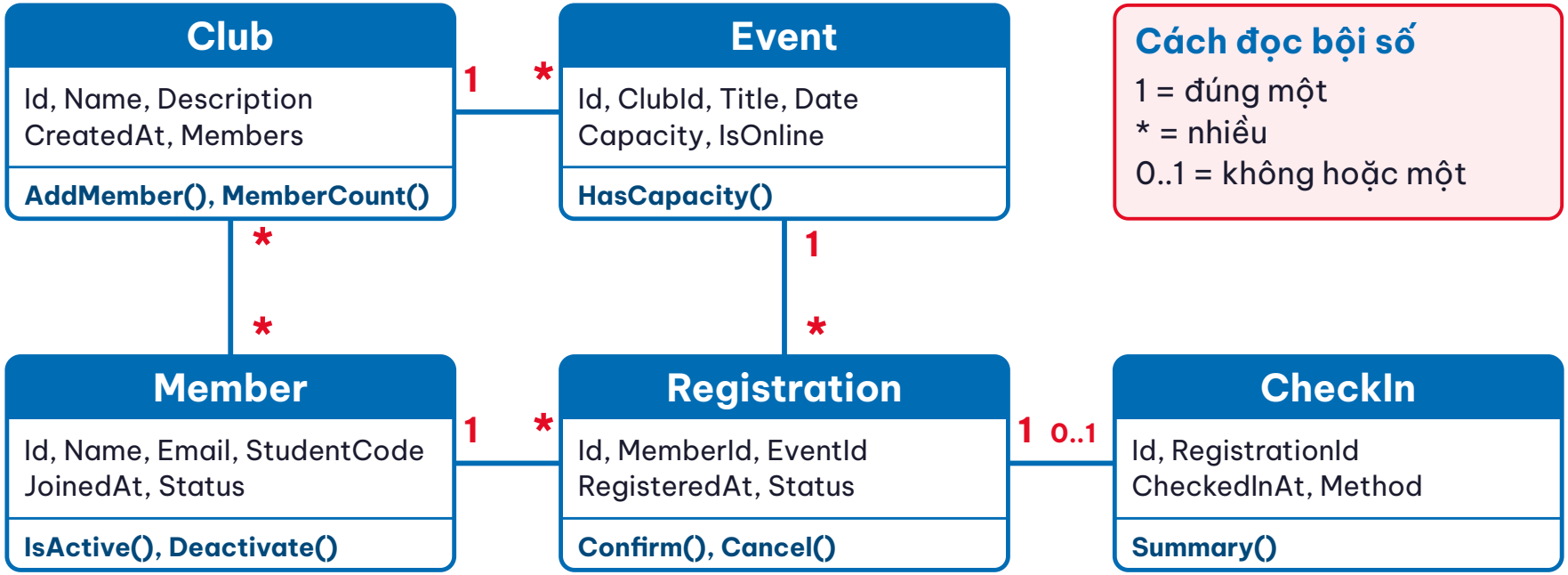
Thuộc tính: `Id, MemberId, EventId, RegisteredAt, Status`
Hành vi (method): `Confirm()`, `Cancel()` — đổi trạng thái theo vòng đời

✅ CheckIn — điểm danh

Thuộc tính: `Id, RegistrationId, CheckedInAt, Method`
Hành vi (method): `Create()` (chỉ khi Confirmed), `Summary()`

Quy ước: `private set` (chỉ class tự sửa) · constructor private (không cho `new` từ ngoài) · tạo qua `Create()`

4.5 Sơ đồ lớp sơ bộ (class diagram)



Hình 1.2. Sơ đồ lớp sơ bộ của EduHub (tự vẽ).

4.6 Quan hệ và khoá ngoại (FK)

QUAN HỆ	LOẠI	KHOÁ NGOẠI (FK)
Member — Club	* — *	bảng trung gian `ClubMember` (làm ở Ch04)
Club — Event	1 — *	`Event.ClubId`
Member — Registration	1 — *	`Registration.MemberId`
Event — Registration	1 — *	`Registration.EventId`
Registration — CheckIn	1 — 0..1	`CheckIn.RegistrationId`

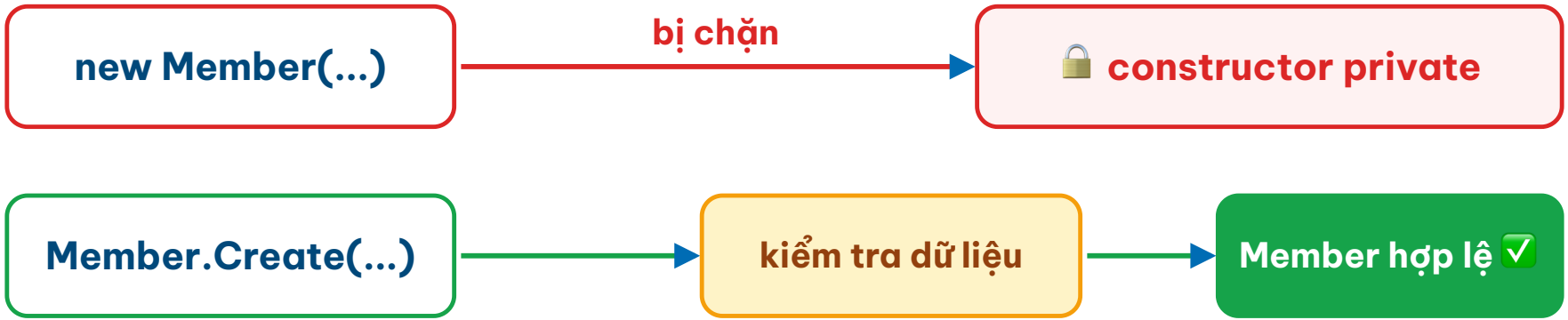
Khoá chính (PK) = mã riêng của mỗi entity: `Id` kiểu `Guid` (chuỗi mã ngẫu nhiên, không trùng).

Code Member — thuộc tính & constructor

```
public class Member
{
    public Guid Id { get; private set; }
    public string Name { get; private set; }
    public string Email { get; private set; }
    public string StudentCode { get; private set; }  // mã số sinh viên
    public DateTime JoinedAt { get; private set; }
    public MemberStatus Status { get; private set; }
}
```

Mỗi dòng là một **thuộc tính** (property). `private set` = chỉ trong class mới được sửa giá trị.

4.8 Vì sao constructor (hàm khởi tạo) là private?



Chỉ có **một cửa vào**: `Create()`. Nó kiểm tra trước khi tạo, nên Member sai dữ liệu không thể xuất hiện.

Code Member — factory Create()

```
public static Member Create(
    string name, string email, string studentCode)
{
    if (string.IsNullOrWhiteSpace(name))   // tên không được trống
        throw new ArgumentException("Tên trống");
    if (string.IsNullOrWhiteSpace(email) || !email.Contains('@'))
        throw new ArgumentException("Email sai");
    // ... kiểm tra mã số sinh viên tương tự ...
    return new Member(name.Trim(),
        email.Trim().ToLowerInvariant(), studentCode.Trim());
}
```

Factory = hàm "nhà máy": kiểm tra rồi mới tạo object (đối tượng). `throw` = báo lỗi và dừng.

Code Club — AddMember()

```
public void AddMember(Member member)
{
    ArgumentNullException.ThrowIfNull(member);    // không được null
    if (_members.Any(m => m.Id == member.Id))     // trùng Id?
        throw new InvalidOperationException(
            $"{member} đã là thành viên");
    _members.Add(member);
}

public int MemberCount() => _members.Count;
```

Muốn thêm thành viên phải đi qua `AddMember()`.

Code Event — Create() & HasCapacity()

```
public static Event Create(Guid clubId, string title,
    DateTime date, int capacity, bool isOnline = false)
{
    if (string.IsNullOrWhiteSpace(title))   // tiêu đề không được trống
        throw new ArgumentException("Tiêu đề trống");
    if (capacity <= 0)   // sức chứa phải lớn hơn 0
        throw new ArgumentOutOfRangeException(nameof(capacity));
    return new Event(clubId, title.Trim(), date, capacity, isOnline);
}

public bool HasCapacity(int registeredCount) =>
    registeredCount < Capacity;    // còn chỗ nếu chưa đủ
```

Code Registration — Confirm()

```
public void Confirm()
{
    if (Status != RegistrationStatus.Pending)
        throw new InvalidOperationException(
            $"Không thể xác nhận ({Status})");
    Status = RegistrationStatus.Confirmed;
}

public void Cancel() { … }   // cho phép khi Pending hoặc Confirmed
```

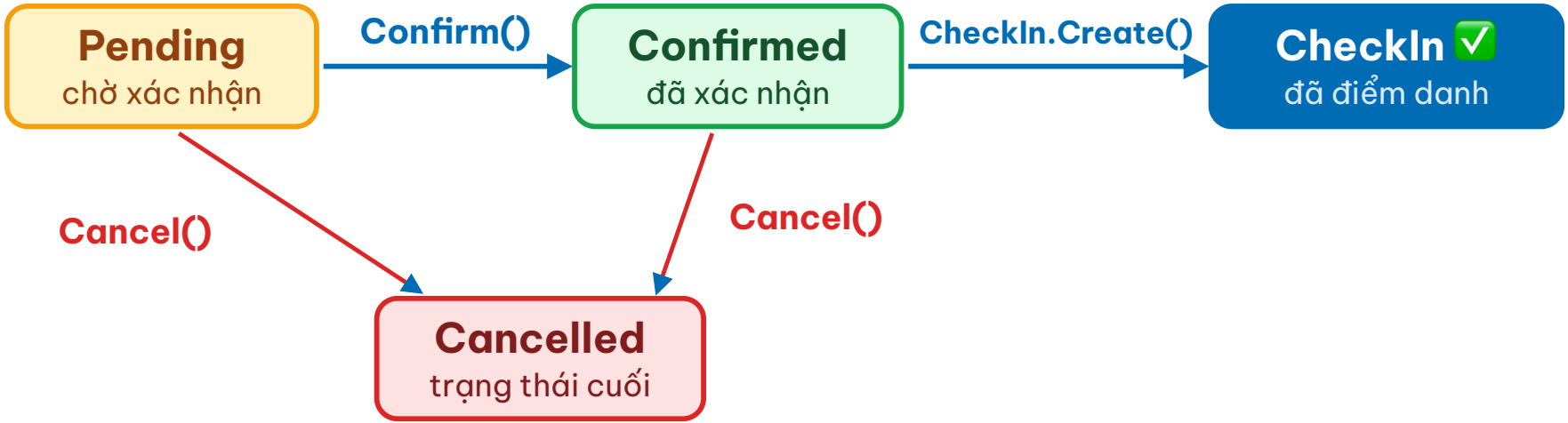
Chỉ phiếu **Pending** mới được xác nhận.

Code CheckIn – Create()

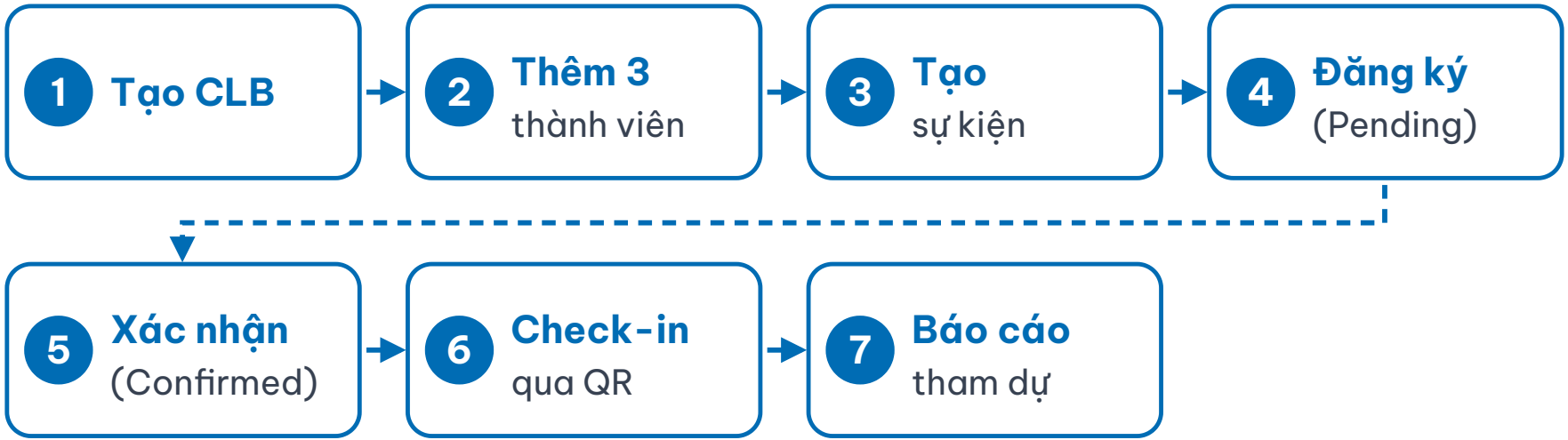
```
public static CheckIn Create(
    Registration registration,
    string method = CheckInMethod.Manual)
{
    ArgumentNullException.ThrowIfNull(registration);
    if (registration.Status != RegistrationStatus.Confirmed)
        throw new InvalidOperationException(
            "Chỉ phiếu Confirmed mới check-in");
    return new CheckIn(registration.Id, method);
}
```

Chưa xác nhận thì **chưa được** check-in.

4.14 Vòng đời của Registration



Console demo — 7 bước



5

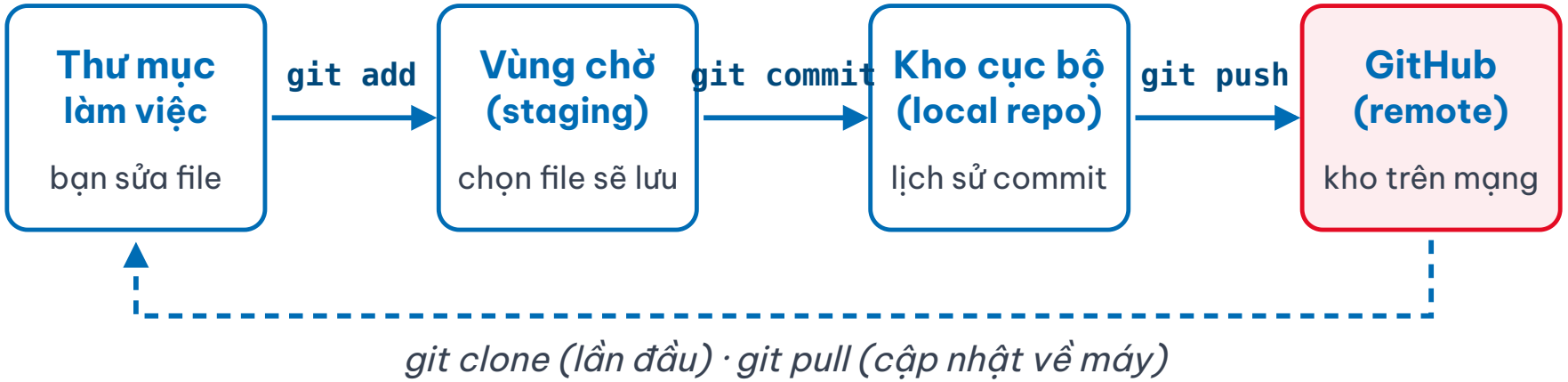
PHẦN 5 / 6

PHẦN 5

1.5 · Git và GitHub cơ bản

- Git và GitHub: bốn khu vực của một repo
- Clone, branch, commit, push
- Pull Request
- Quy ước commit message (Conventional Commits)

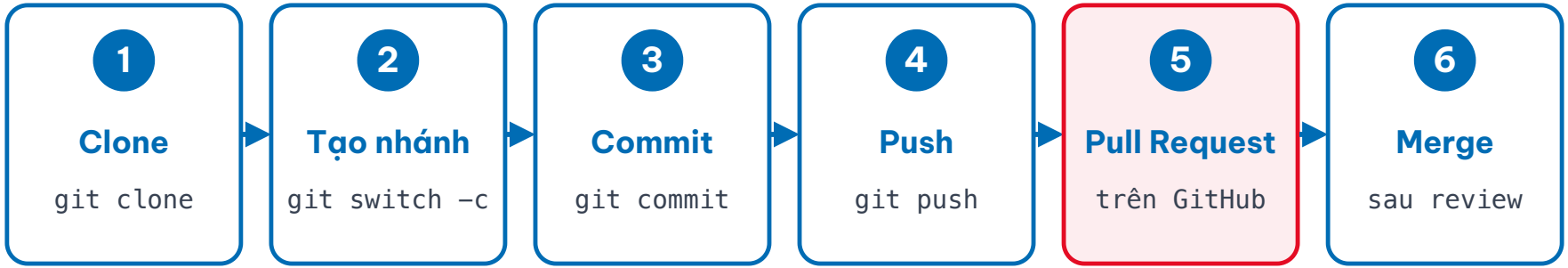
Git và GitHub: bốn khu vực của một repo



Hình 1.3. Luồng đi của code từ máy bạn lên GitHub (tự vẽ).

Git lưu lịch sử thay đổi trên máy bạn. **GitHub** là nơi lưu repo trên mạng để làm việc nhóm và nộp bài.

Quy trình làm việc với repo



Hình 1.4. Sáu bước từ clone đến merge (tự vẽ).

ACTIVE RECALL Vì sao làm việc trên nhánh riêng thay vì sửa thẳng `main`?

```
# 1. Lấy repo về máy (làm một lần)
git clone https://github.com/<tài-khoản>/eduhub.git
# 2. Tạo nhánh riêng cho bài tập
git switch -c ch01/DH24001
# 3. Sửa code, chọn file rồi lưu thành commit
git add .
git commit -m "feat(domain): add Member entity"
# 4. Đẩy nhánh lên GitHub
git push -u origin ch01/DH24001
```

Kiểm tra: `git status` (đang thay đổi gì), `git log` (lịch sử).

5.4 Branch — nhánh làm việc riêng



Hình 1.5. Nhánh tách từ main rồi gộp lại bằng PR (tự vẽ).

- `main` luôn là bản ổn định; mỗi bài tập hoặc tính năng làm trên **một nhánh riêng**
- Gộp nhánh về `main` bằng **Pull Request**, không sửa thẳng vào `main`

Pull Request — đề nghị gộp code

Pull request #1 — Open

feat(domain): add EduHub domain classes

`ch01/DH24001 → main`

Mô tả (description)
What: thêm 5 class domain của EduHub
Why: hoàn thành bài tập Ch01 (HĐ1.3)
How to test: chạy dotnet run, xem báo cáo

Reviewers
Giảng viên

Commits
3 commit

Hình 1.6. Minh hoạ trang Pull Request trên GitHub (tự vẽ).

PR cho giảng viên đọc và góp ý trước khi gộp vào `main`. Mô tả nêu **what, why, how to test**.

5.6 Quy ước commit message

```
type(scope): subject          # Conventional Commits
feat(domain): add Member.Create with validation
```

TYPE	DÙNG KHI
feat	thêm tính năng mới
fix	sửa lỗi
docs	chỉ sửa tài liệu (README, sơ đồ)
refactor	sắp xếp lại code, không đổi hành vi
test · chore	thêm test · việc vặt (cấu hình, đổi tên file)

Subject viết ngắn, thể mệnh lệnh, bằng tiếng Anh.

Commit message tốt và chưa tốt

❌ `update`, `fix bug`, `final final` → ✅ `fix(club): reject duplicates`

❌ Một commit chứa cả bài, nộp phút chót → ✅ Mỗi tính năng một commit, commit dần

❌ Pull Request không có mô tả → ✅ Mô tả đủ what, why, how to test

5.8 Thực hành: Pull Request đầu tiên

1. Tạo repo `eduhub` trên GitHub và **clone** về máy
2. Tạo nhánh `ch01/<MSSV>`
3. Thêm `README.md` giới thiệu bản thân, **commit** theo quy ước
4. **Push** nhánh lên GitHub
5. Mở **Pull Request** về `main`, tag giảng viên



PHẦN 6 / 6

PHẦN 6

Bài tập & tổng kết

- Bài tập Ch01 — Phân tích yêu cầu và Domain Modeling EduHub
- Cách nộp bài và lỗi thường gặp
- Demo đáp án mẫu
- Tổng kết và câu hỏi tự kiểm tra

Bài tập Ch01 — 100 điểm

1	2	3	4	5
User Stories	**Sơ đồ lớp**	**Domain classes**	**Console demo**	**Git và PR**
20 điểm	15 điểm	35 điểm	20 điểm	10 điểm

Bài tập Ch01 — Phân tích yêu cầu và Domain Modeling EduHub. Chấm theo **HĐ1.3** (bài tập chương), liên quan CLO1, CLO4 và CLO5.

Yêu cầu từng nhiệm vụ

- **1 · User Stories** — ít nhất 5 story, mỗi story có ít nhất 2 tiêu chí nghiệm thu, đủ 3 vai trò
- **2 · Sơ đồ lớp** — 5 class với thuộc tính, quan hệ và khoá ngoại; vẽ bằng Mermaid hoặc draw.io
- **3 · Domain classes** — `private set`, constructor private, `Create()`, có kiểm tra dữ liệu
- **4 · Console demo** — `Program.cs` chạy đủ luồng nghiệp vụ và in báo cáo tham dự
- **5 · Git và PR** — ít nhất 3 commit theo Conventional Commits, Pull Request có mô tả

Cách nộp bài

🌿 Làm trên nhánh riêng `ch01/<MSSV>` của repo `eduhub`

📝 Ít nhất **3 commit**, message theo **Conventional Commits**

📬 Mở **Pull Request** về `main`, tag giảng viên, mô tả đủ what, why, how to test

▶️ `dotnet run` chạy không lỗi; README có ảnh chụp kết quả

> Đề và rubric chi tiết: `02_Projects/ch01/` · Bài tập là thành quả cá nhân.

Lỗi thường gặp

❌ Mọi thuộc tính để `public set` → ✅ `private set` + method hành vi

❌ Thiếu khoá ngoại (`Event` không có `ClubId`) → ✅ Vẽ sơ đồ lớp trước; phía "nhiều" giữ FK

❌ `List<Member>` public, ai cũng thêm/xoá được → ✅ `IReadOnlyList` + `AddMember()`

❌ Chuỗi `"Pending"` rải khắp nơi → ✅ Dùng hằng số hoặc enum

Demo (1/2) — Chạy đáp án mẫu

```
# Mở thư mục đáp án
cd 02_Projects/ch01/solution/EduHub.Ch01
# Chạy chương trình
dotnet run
```

- Code nằm trong `Domain/`: 5 file, mỗi file một entity
- `Program.cs` chạy 7 bước như slide trước, thêm bước 8: thử vi phạm luật nghiệp vụ

HỎI SV Vì sao tạo đối tượng bằng `Club.Create(...)` mà không dùng `new Club()`?

Demo (2/2) – Quy tắc nghiệp vụ

```
// Phiếu đã huỷ thì không thể xác nhận lại
registrations[1].Cancel();
try { registrations[1].Confirm(); }     // thử xác nhận lại
catch (InvalidOperationException ex)    // bị báo lỗi thì vào đây
{
    Console.WriteLine(ex.Message);        // in lời báo lỗi
}
```

```
Không thể xác nhận phiếu đang ở trạng thái Cancelled.
```

HỎI SV Quy tắc này nên nằm ở `Registration` hay ở `Program.cs`? Vì sao?

Tổng kết Chương 1

- **.NET 10 LTS** + VS Code / Rider: cài và chạy được chương trình đầu tiên
- **OOP**: đóng gói, kế thừa, đa hình, interface; học cách áp dụng, không học thuộc
- **EduHub**: User Story ➜ class ➜ sơ đồ lớp ➜ code C# (`Create()`, `private set`)
- **Git/GitHub**: clone, branch, commit, push, Pull Request; commit theo Conventional Commits

Câu hỏi tự kiểm tra

✔ CHECKPOINT — CHƯƠNG 1

1. Lệnh nào tạo project console mới và lệnh nào chạy nó?
2. Vẽ lại sơ đồ lớp của EduHub từ trí nhớ: 5 class và quan hệ.
3. Viết class `Member` có 4 thuộc tính. Kiểm tra `Email` đặt ở đâu?
4. Nêu thứ tự các bước từ clone đến Pull Request.
5. Viết commit message đúng quy ước khi thêm `HasCapacity()`.

Chuẩn bị cho Chương 2 (tuần 3–4)

🧱 **Chương 2:** 5 nguyên lý SOLID, các Design Pattern (Factory, Decorator, Strategy, Observer), unit test đầu tiên

🌿 **Git nâng cao:** rebase, xử lý conflict, code review trên Pull Request

📖 Đọc tài liệu [5] *Design Patterns* (GoF) và [6] *GitHub Skills*

💻 Hoàn thiện bài tập Ch01 và nộp Pull Request đúng hạn